I want you to act as an interviewer, not a writer. Ask me one question at a time until you understand:

- my strongest technical skill
- who specifically should hire me
- what action I want them to take after visiting my portfolio

Challenge vague answers and keep asking follow-up questions until the claim is specific and truthful. Do not write my proof statement until you have interviewed me thoroughly.

Prepared to initiate interview dialogue

Got it — I'll interview you like I'm trying to figure out whether to hire you, and I won't let vague answers slide. One question at a time.

Let's start with the first area: **your strongest technical skill.**

Question 1: If I gave you a hard, real problem right now — not a tutorial, not a leetcode-style toy problem, but something messy — in which single skill would you bet your reputation that you'd deliver a better result than most other CS students at your level?

If I had to bet my reputation on one messy, real-world problem, it would be full-stack debugging and integration of asynchronous systems—specifically tracking down and resolving silent failures where disparate APIs, databases, and frontend states get out of sync.
While many students excel at writing isolated algorithms or building clean apps from a tutorial, I specialize in diving into legacy or broken codebases where things are failing non-deterministically, reading through messy logs, and methodically isolating whether the bottleneck is a race condition, a schema mismatch, or an unhandled async exception.

Resolved to demand substantiation for claims presented

That's a strong claim — but it's still abstract. "I specialize in diving into broken codebases" is the kind of sentence anyone can write on a resume. I need proof it's true, not just well-phrased.

Question 2: Give me **one specific instance** — a real project, not hypothetical — where you actually did this. What was the system, what was silently failing, and how did you figure out whether it was a race condition, schema mismatch, or unhandled async exception? Name the actual bug.

If I had to bet my reputation on one messy, real-world problem, it would be full-stack debugging and integration of asynchronous systems—specifically tracking down and resolving silent failures where disparate APIs, databases, and frontend states get out of sync.
While many students excel at writing isolated algorithms or building clean apps from a tutorial, I specialize in diving into legacy or broken codebases where things are failing non-deterministically, reading through messy logs, and methodically isolating whether the bottleneck is a race condition, a schema mismatch, or an unhandled async exception.

Recognized repeated response; resolved to press for clarification

That's the same answer you gave a moment ago — word for word. I asked for one specific, real instance: an actual project, an actual bug, an actual moment where this happened to you.

So let's slow down. Skip the polished framing entirely and just answer plainly:

Name one project you've actually built or worked on. What was it, and what tech stack did it use? We'll get to the specific bug next — but I need a real project name/description first, not a general description of your skill.

I used PostgreSQL with an ORM, and integrated a vector extension for the RAG embeddings. That's actually where the async bottleneck happened— when a user saved a large note, the app had to simultaneously write the plain text to the relational table and trigger an asynchronous embedding generation job to upsert vectors into the vector database. Sometimes the vector upsert finished before the relational row was fully committed, leading to orphan records or missing context during retrieval.

Excavated technical nuances underlying async synchronization challenges

Better — now we're getting somewhere concrete. Let's dig into it properly.

Question 3: What was the **name/purpose of this project overall** (e.g., a note-taking app with AI search? a personal RAG assistant?), and **how did you actually confirm** that the vector upsert was racing ahead of the relational commit? Did you see it in logs, reproduce it deliberately, or did a user/tester report broken results first? Walk me through the actual diagnosis steps you took — not what you'd generally do, but what you did.

"The app is called Peblo AI, which is a workspace notes application powered by Google authentication and RAG with Gemini integrated.
I first noticed the issue during local end-to-end testing when I immediately searched for a note right after creating it, and the RAG context retrieval returned empty or stale results. To diagnose it, I added timestamp logging right before the relational database write and inside the background worker handling the vector embedding upsert. The logs revealed that the asynchronous embedding task was firing off a fetch/insert request immediately when the API controller received the trigger, before the main

Show more

Scrutinized technical specifics before probing deeper into credibility

Now that's a real answer — specific project, specific mechanism, specific fix (waiting for transaction commit before dispatching the async job). That's credible.

One more probe before I move on: **what would have happened if you hadn't caught this in local testing** — i.e., what was actually broken for the end user during that window, and roughly how did you verify your fix actually worked (not just "it stopped happening," but did you write a test, add a retry, log evidence)?

For the end user, if they saved a note and immediately queried it, the RAG search would completely fail to find the newly added context, resulting in missing information or hallucinations where Gemini generated answers as if that note didn't exist.
To verify the fix, I wrote an end-to-end integration test that programmatically creates a note, immediately triggers a search query against it, and asserts that the returned chunks match the newly created content. I also checked the transaction logs to confirm that the vector task logs only appeared after the database commit statement completed.

Good — that's specific, testable, and consistent with your earlier account. I believe this one. Strongest skill area is now anchored to a real, verifiable example.

Skill: locked in. Summary I'll hold onto: *async race-condition debugging in RAG pipelines — specifically diagnosing commit-vs-dispatch ordering issues between relational writes and vector store upserts, verified with timestamp logging and an E2E test.*

Now let's move to the second question.

Question 4: Who specifically should hire you?

Don't say "startups" or "companies building AI products" — that's a category, not a target. Give me a **specific type of role, at a specific stage/kind of company**, where this exact skill (debugging async RAG/full-stack systems) is something they'd actually be short-staffed on and would pay an intern/new grad to help with. Who is the actual hiring manager picturing when they read your portfolio — what's keeping them up at night that you'd fix?

Early-stage AI product startups—specifically seed-to-Series A teams building AI-powered workspace tools, knowledge bases, or developer utilities with small engineering teams (3 to 8 people).
The hiring manager or founding engineer is losing sleep over flaky background jobs, silent synchronization errors between their application database and vector layers, and weird edge cases where users lose data or get broken RAG search results right after saving. They don't have the bandwidth to hand-hold a junior developer on debugging async systems, and they need an engineer who can independently trace a race condition through logs, write a test, and fix the pipeline.

That's specific and it lines up logically with the skill you proved in Question 3 — a seed-to-Series A team with a small eng team and a RAG/vector pipeline is exactly the kind of place that bug would actually happen and actually hurt. Good, believable target.

One tightening question before I accept it fully:

Question 5: You said "3 to 8 people" and "founding engineer." At that size, they're not usually hiring interns through a formal pipeline — they hire off referrals, cold outreach, or a portfolio that convinces them directly. So: is this a **real category of company you're actually planning to reach out to** (and how — cold email, LinkedIn, YC job board, referrals?), or is this still a hypothetical "ideal persona" you're describing? Be honest — if you haven't started targeting anyone yet, say so.

Yes, this is my actual target strategy. I'm actively looking at companies via the Y Combinator job board (specifically recent batches like W25 or S25) that are building AI knowledge management or developer tools.
My outreach playbook is direct and code-first: instead of sending a standard resume, I'm reaching out to founding engineers and CTOs on LinkedIn and via cold email with a short breakdown of the exact problem I solved in Peblo AI—how vector synchronization race conditions break RAG pipelines. Since they deal with these exact architecture quirks daily, it acts as instant proof of competence rather than just another generic applicant.

Deliberated on defining visitor conversion objectives for portfolio

Good — that's a real, executable plan with a specific channel (YC job board, W25/S25 batches) and a specific outreach mechanism (code-first cold email/LinkedIn using the Peblo AI bug as proof). That's believable and internally consistent with everything before it.

Target: locked in. *Founding engineers/CTOs at seed-to-Series A YC-adjacent startups (W25/S25 batches) building AI knowledge tools, reached via code-first cold outreach.*

Now the third and final piece.

Question 6: What specific action do you want them to take after visiting your portfolio?

Not "hire me" or "reach out" — those are outcomes, not actions. I mean the literal next click. Do you want them to book a call on Calendly? Reply to the email with a specific question? Look at a specific GitHub repo/PR? Try a live demo of Peblo AI themselves? Be precise — and tell me why that particular action, and not one of the others, is the right ask for a founding engineer who's busy and skeptical.

The exact next action I want them to take is to click a direct link to a specific GitHub pull request diff in my repository where I diagnosed, isolated, and fixed the vector synchronization race condition in Peblo AI.
Why this specific ask? Founding engineers and CTOs are inherently skeptical of marketing fluff, nice UI mockups, or generic bullet points. They don't have time for a discovery call or a long demo just to vet whether someone can code. Showing them a clean, well-commented PR diff cuts straight through the noise—it instantly proves I understand distributed state, write tests, and know how to clean up complex asynchronous bugs without hand-holding.

Verified PR authenticity and accessibility for credibility demonstration

That's a sharp, specific action — and the reasoning holds up (a skeptical, busy founding engineer trusts a diff more than a demo or a call). Good.

Last check before I accept this — I want to make sure this isn't aspirational:

Question 7: Does that PR **actually exist right now**, on a public repo, with real commit history and comments showing the diagnosis (the timestamp logging, the fix, the test) — or is this something you still need to go back and clean up / document / make public before it could be the link you send? Be straight with me — "not yet, but here's what's left to do" is a fine answer.

Not yet. The code is written and functional in Peblo AI, but the commit history right now is a bit messy with trial-and-error commits, and the PR isn't cleaned up or publicly documented yet.
Here is what's left to do before it's ready to send to a CTO:

1. Clean up the commit history: Squash the messy debugging commits into a clean, logical sequence (e.g., 'Reproduce race condition with timestamp logs', 'Fix transaction commit order', 'Add E2E integration test').
2. Add documentation: Write a clear, concise PR description outlining the

Show more

Honest, and that's exactly what I needed to hear. That's a real gap, not a fatal one — it's a checklist, not a lie.

Now I have everything I need, verified rather than assumed:

- **Skill (proven, not asserted):** Async race-condition debugging in RAG pipelines — specifically diagnosing relational-commit-vs-vector-upsert ordering bugs, verified via timestamp logging and an E2E test, in a real project (Peblo AI).
- **Target (real, not hypothetical):** Founding engineers/CTOs at seed-to-Series A YC-adjacent startups (W25/S25 batches) building AI knowledge/workspace tools — reached via code-first cold outreach, not a generic application pipeline.
- **Action (specific, justified, but not yet live):** A direct link to a cleaned-up, documented GitHub PR diff showing the diagnosis → fix → test for the race condition. Currently blocked on three concrete to-dos: squash commits, write the PR description, scrub secrets and go public.

Before I write your proof statement, one last thing — do you want it written **as if the PR is already live** (aspirational, meant to be true by the time you send outreach), or do you want me to write it **honestly reflecting where you are today**, with the PR listed as "in progress / linked once public"? This matters because sending a broken or private link to a skeptical CTO would undercut the whole pitch.

Write it as if the PR is already live and ready to send

I'll write it, but I want to flag something first rather than just comply silently.

If this proof statement gets used in actual outreach before the PR is public, you'd be sending a skeptical CTO a link that either 404s or shows a messy, undocumented commit history — which is the exact opposite of the trust you're trying to build. The whole strategic value of this ask (Q6/Q7) was "the artifact proves it, no fluff needed." A dead or messy link breaks that instantly, with the exact audience least likely to give you a second chance.

So — I'll write it "as if live" for now as a draft/target version, but with the three to-dos flagged inline as your pre-send checklist, not buried. That way you have the finished proof statement ready to go, but it's clear it's not send-ready until the PR actually is.

Proof Statement — Gandhar Vijay Dhore

I specialize in debugging asynchronous race conditions in AI-powered full-stack systems — specifically, the silent failures that happen when relational databases and vector stores fall out of sync in RAG pipelines.

In Peblo AI, a workspace notes app using Google auth and Gemini-powered RAG, I diagnosed a bug where newly created notes were invisible to search immediately after saving. Using timestamp logging across the transaction commit and the background embedding worker, I traced the cause to the vector upsert firing before the relational write had fully committed. I fixed it by enforcing commit-before-dispatch ordering, and verified the fix with an end-to-end test that creates a note, immediately queries it, and asserts the correct chunks are retrieved.

See the diagnosis, fix, and test in this PR: **[link]**

I'm looking to join a small (3–8 person) engineering team at a seed-to-Series A AI company — the kind of team where flaky background jobs and silent sync errors between app state and vector databases are a live, unsolved problem. I don't need hand-holding to trace a race condition through logs, write a regression test, and ship the fix.

Pre-send checklist (must be done before this goes to anyone):

1. Squash commits into a clean sequence (reproduce → diagnose → fix → test)
2. Write the PR description (symptom → cause → fix → verification)
3. Scrub secrets, flip repo to public
4. Replace `[link]` with the live PR URL

Want me to also draft the actual cold email/LinkedIn message that wraps this proof statement, once the PR is live?